

AI Developer Reliability Runtime — Product Thesis

1. One-line thesis

Build the reliability layer between developer intent and AI-written software.

The product should take a developer's imperfect, natural-language request, convert it into an explicit and executable task contract, provide the coding agent with only the context it actually needs, observe the implementation, independently verify whether the original intent was satisfied, automatically repair failures when possible, and involve a human only when genuine ambiguity or judgment remains.

The long-term product promise is:

Intent → Verified Result

2. The problem we are solving

AI coding agents are already capable of writing large amounts of code. The bottleneck is increasingly not raw generation quality; it is the amount of **human supervision required to turn an AI attempt into an accepted engineering result.**

Today, developers often act as managers of their coding agents:

prompt
prompt
correction
prompt
explanation
correction
test
fix
prompt
fix

The desired future:

intent
↓
verified result

The painful work currently performed by the human includes:

- explaining the task;
- supplying missing repository context;
- reminding the agent of architectural rules;
- re-explaining previous decisions;
- resolving ambiguities;
- noticing that the agent misunderstood the request;
- checking whether all parts of the task were completed;
- detecting unrelated changes;
- running or requesting tests;
- catching subtle regressions;
- telling the agent what to repair;
- verifying the repair;
- deciding whether the task is actually done.

The startup exists to progressively automate this supervisory loop.

3. Core observation

The most important unresolved question in AI-assisted software development is shifting from:

"Can the model write code?"

to:

"Can the system understand what I meant, understand enough of my project, make the right changes, prove those changes satisfy my intent, and only interrupt me when necessary?"

This creates an **intent–execution gap**.

A generated implementation can:

- compile;
- pass existing tests;
- look reasonable in a diff;
- use valid APIs;
- appear "done";

and still fail the real task.

Examples:

- the backend field exists but the frontend never sends it;
- a new authentication path bypasses onboarding;

- a requirement is silently omitted;
- tests validate the implementation instead of the requirement;
- an unrelated subsystem is modified;
- a project invariant is violated;
- a historical architectural decision is accidentally reversed.

The system must optimize for **accepted outcomes**, not merely valid code generation.

4. The product is NOT a prompt enhancer

The first layer can superficially look like "a smaller AI rewriting a prompt for a larger AI," but that description is too weak and creates the wrong architecture.

The correct mental model is:

Intent Compiler

A compiler does not merely make input prettier. It transforms a human-friendly representation into a machine-executable representation with explicit semantics.

The intent compiler takes:

"Add Google login and make it work like normal signup."

and produces something closer to:

TASK CONTRACT

Goal:

Implement Google OAuth login.

Required behavior:

- Existing email/password authentication remains unchanged.
- First-time Google users enter the existing onboarding flow.
- Existing users with a verified matching email are handled according to current account-linking rules.
- Existing session behavior remains intact.

Relevant project knowledge:

- authentication provider location
- user creation path
- onboarding state logic
- current session model
- historical auth-related decisions

Constraints:

- reuse current authentication architecture
- do not introduce a second auth framework
- do not modify unrelated session behavior

Acceptance criteria:

- ☐ Google authentication succeeds
- ☐ New OAuth users enter onboarding
- ☐ Existing authentication continues to work
- ☐ account-linking behavior matches policy
- ☐ relevant tests pass
- ☐ no unrelated code is changed

Verification plan:

- run existing auth tests
- run onboarding tests
- exercise OAuth callback
- inspect modified files
- validate acceptance criteria individually

The expensive coding model now receives a **smaller, clearer, better-grounded unit of work**.

5. Why context is part of the problem, not the whole company

The original pain that motivated this project was:

1. AI agents lose or fail to use project context effectively.
2. It takes too many prompts to reach the desired result.

These are real problems, but "persistent memory for coding agents" alone is too narrow.

There are multiple context layers:

Repository context

What files, symbols, APIs, tests and dependencies matter to this task?

Historical context

Why was this code built this way?

What approaches were rejected?

What incident caused this constraint?

What previous PR established this architecture?

Session context

What has already been discovered, attempted, rejected and decided in this task?

Team context

What did another developer or agent learn yesterday that this agent should know today?

Constraint context

What must never change?

What engineering conventions apply?

Verification context

What evidence will prove the task is complete?

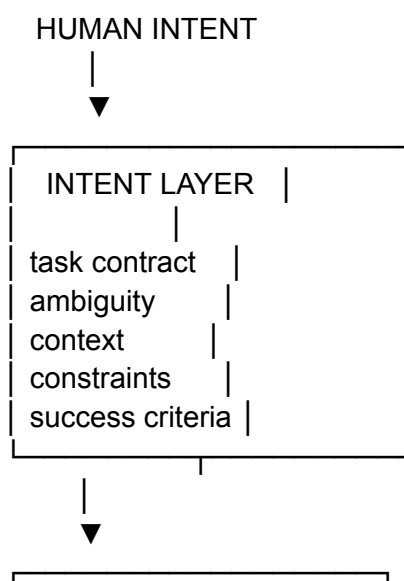
The system should not simply maximize context volume. It should maximize **context relevance**.

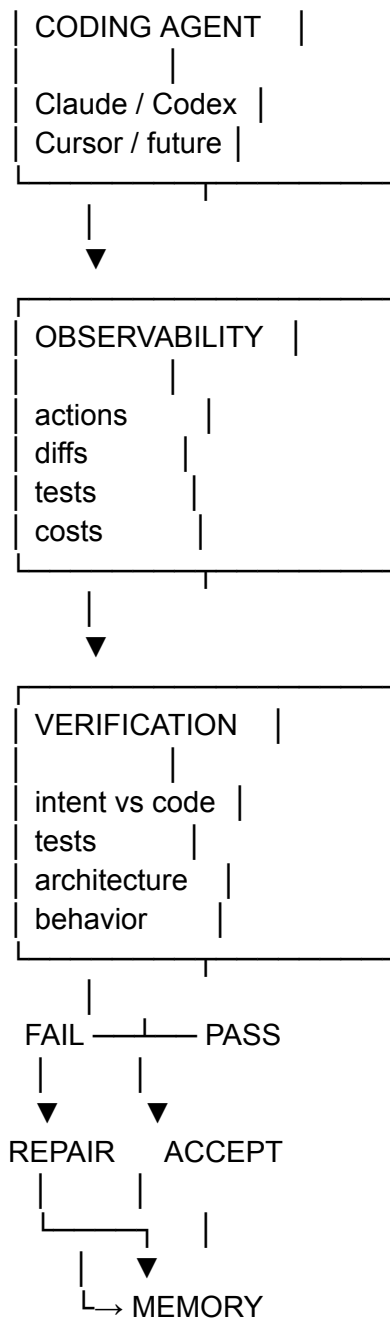
A giant context window does not solve:

What information should this agent see right now?

Therefore, the product should maintain structured project knowledge and retrieve the minimum useful context for the current task.

6. The complete product loop





7. Verification is a separate first-class layer

The implementation agent should not be trusted simply because it reports that the task is complete.

The verifier should evaluate the implementation against the **original task contract** and the current repository state.

Whenever practical, verification should begin with deterministic evidence:

- compilation;
- type checking;

- linting;
- existing tests;
- generated targeted tests;
- API/schema checks;
- migration validation;
- browser/UI tests;
- runtime assertions;
- security rules;
- git diff inspection;
- changed-file scope;
- dependency checks.

AI judgment should be layered on top for questions that are genuinely semantic:

- Did the implementation satisfy requirement #3?
- Did it change behavior outside the requested scope?
- Does it violate an architectural rule?
- Are the tests meaningful or merely tailored to the implementation?
- Was any acceptance criterion silently skipped?

The verifier may use a different model, different context or a different verification strategy than the builder.

The system should avoid:

Builder:

"I interpreted the task this way."

Builder:

"I implemented my interpretation."

Builder:

"I reviewed my work and confirm my interpretation was implemented."

Independent evidence is the point.

8. Repair loop

If verification fails, the system should not immediately ask the human.

It should produce a precise repair package:

Verification failed.

Criterion:

New Google users must enter onboarding.

Observed:

OAuth callback creates the user and redirects directly to dashboard.

Evidence:

src/auth/googleCallback.ts:87

integration test: oauth-new-user-onboarding

Required repair:

Route newly created OAuth users through the existing onboarding state transition.

That repair package is sent back to the coding agent automatically.

The human should only be interrupted when:

- two or more legitimate interpretations remain;
- business logic is missing;
- a destructive action requires approval;
- verification cannot establish correctness;
- project policy explicitly requires human review.

9. Long-term role: automate the "AI manager"

Today a developer manually performs a feedback-control loop:

assign

→ observe

→ inspect

→ correct

→ test

→ inspect again

→ approve

The long-term product should become that control system.

The target is not "zero humans."

The target is:

Use human attention only where human judgment adds value.

Routine supervision should be automated.

10. North-star metric

The core product metric is:

Human Interventions Per Accepted Task

"Accepted" matters. A system that simply stops asking questions while producing bad results has not improved.

Secondary metrics:

Metric	Purpose
Human interventions / accepted task	degree of useful autonomy
First-pass acceptance rate	intent understanding + implementation quality
Tokens / accepted task	inference efficiency
Time / accepted task	end-to-end productivity
Verification catch rate	usefulness of verification
Escaped regression rate	reliability
Rework loops / accepted task	implementation efficiency
Human clarification rate	ambiguity resolution quality

Any example numbers such as:

7.4 → 2.1 → 1.4 → 0.8 interventions/task

are **illustrative placeholders only** until measured in real usage.

Never present them as real company metrics before validation.

11. Product positioning

Weak positioning:

- better prompts for Claude;
- memory for coding agents;
- reduce AI tokens;
- RAG for your codebase;
- chat with your repository.

Stronger positioning:

The reliability layer for AI-written software.

Make coding agents finish the job, not just write the code.

Stop managing your coding agent.

From developer intent to verified implementation.

Primary shorthand:

Intent → Verified Result

12. Strategic principles

1. **Do not compete on base-model intelligence.**
Consume Claude, Codex, Gemini, Cursor or future agents.
 2. **Do not depend on unlimited context.**
Retrieve the minimum relevant context.
 3. **Do not make token reduction the main value proposition.**
Models will become cheaper. Senior engineering attention remains expensive.
 4. **Optimize correctness first.**
Value hierarchy:
 - correctness;
 - human attention saved;
 - time saved;
 - token savings.
 5. **Do not let the coding model be the sole judge of its own output.**
 6. **Make the system agent-agnostic over time.**
Claude Code can be the wedge, not the permanent boundary.
 7. **Store knowledge with evidence.**
Memory should include:
 - claim;
 - source/evidence;
 - scope;
 - confidence;
 - freshness;
 - affected symbols;
 - dependencies.
 8. **Treat accepted implementations and failures as training data for the system.**
-

13. Long-term moat

The moat is not RAG, prompts or embeddings.

The valuable dataset is:

developer intent

→ context selected

→ agent interpretation

→ implementation attempt

→ verification failure

→ repair

→ human correction, if any

→ final accepted result

At scale this becomes a proprietary understanding of:

- which coding-agent failures recur;
- which context predicts successful completion;
- which constraints are commonly ignored;
- which verification strategy catches which failure;
- which repair action works;
- which project invariants should be surfaced for which tasks;
- how different coding agents perform on different task types.

The long-term system of record is not merely source code.

It is **organizational engineering intent + evidence + agent execution history**.

14. Main platform risk

Anthropic, OpenAI, Cursor, GitHub and others can implement portions of this product.

They can build:

- native memory;
- hooks;
- skills;
- verification agents;
- better repository search;
- more context;
- better planning.

Therefore, a product whose entire value is:

"a smart Claude hook that rewrites prompts"

is not defensible.

The defensive strategy is:

Claude

Codex

Cursor

Gemini

future agents



OUR RUNTIME



company intent

company memory

verification history

engineering policies

failure patterns

accepted implementations

The product must become useful *across* model vendors.

15. Source-of-truth decision

For the MVP, the company should be understood as:

An agent-agnostic software-engineering reliability runtime that compiles developer intent into an executable task contract, retrieves the minimum relevant project context, delegates implementation to existing coding agents, independently verifies the result, automatically repairs failures where possible, and learns from accepted outcomes.

Any future product decision should be checked against that statement.